

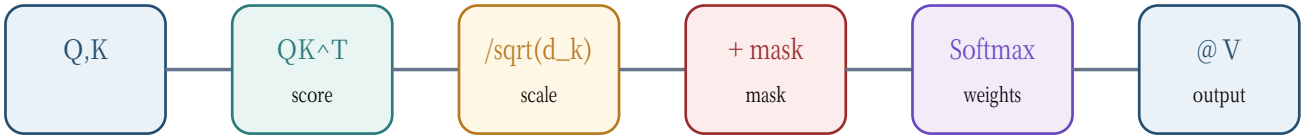
Scaled Dot-Product Attention

从 QK^T 到真正的 Attention 输出

本讲目标

把第 3 讲 Dot Product、第 4 讲 Matrix Multiplication、第 16/17 讲概率与 Softmax、第 53 讲 Q/K/V 第一次全部拼成一条完整计算链。学完后，Attention 不能再是一个黑盒方框。

Scaled Dot-Product Attention 的完整数据流



每一行 Softmax 都在 Key 维度上归一化，最后得到对 V 的加权和

本讲关键问题	学完后应能回答
QK^T	每个元素到底代表什么，Shape 为什么是 $T \times T$
除以 $\text{sqrt}(d_k)$	为什么不是装饰，而是控制 score 尺度
Mask	为什么必须在 Softmax 前处理
Softmax	为什么每个 Query 得到一行权重分布
@ V	为什么最终输出仍是一个 Token 表示

1. 从第 53 讲继续：我们已经有 Q、K、V

$$Q = XW_Q, K = XW_K, V = XW_V$$

本讲为了手算清晰，先忽略 Batch 和 Head，只看 3 个 Token，每个 Q/K 是 2 维：

$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$$

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$$

$$V = \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 3 & 1 \end{bmatrix}$$

因此：T=3，d_k=2，d_v=2。Q/K 负责决定“关注谁”，V 负责提供“真正被读取的内容”。

重要区分

Q/K 的数值决定 Attention Weight；V 的数值不参与匹配分数本身。也就是说“看谁”和“从被看的对象那里取什么”是两步。

2. 第一步： QK^T = 所有 Query 对所有 Key 的 Dot Product

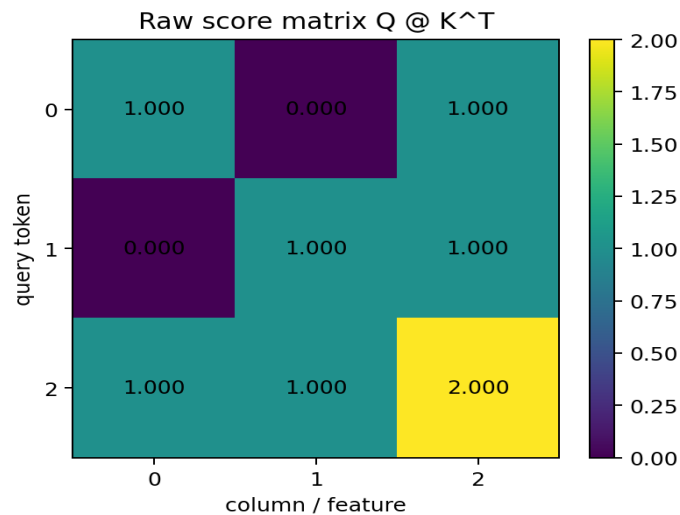
$$\text{Scores} = Q @ K^T$$

因为 $Q:(T,d_k)$, $K^T:(d_k,T)$, 所以：

$$(T,d_k) @ (d_k,T) \rightarrow (T,T)$$

本例中：

$$QK^T = [[1,0,1],[0,1,1],[1,1,2]]$$



元素 $\text{Scores}[i,j]$ 就是第 i 个 Token 的 Query 与第 j 个 Token 的 Key 的 Dot Product。

读矩阵的方法

固定一行 i ：这是“Query i 对所有 Key”的匹配分数。固定一列 j ：这是“所有 Query 对 Key j ”的匹配分数。

3. 手算一个 Score：它仍然只是 Dot Product

例如 Query 3：

$$q_3 = [1, 1]$$

Key 1：

$$k_1 = [1, 0]$$

因此：

$$\text{score}(3, 1) = q_3 \cdot k_1 = 1 \cdot 1 + 1 \cdot 0 = 1$$

再看 Key 3：

$$\text{score}(3, 3) = [1, 1] \cdot [1, 1] = 2$$

所以第三行是 [1, 1, 2]。这里没有新数学，只是一次性把所有 Query-Key Dot Product 用矩阵乘法计算出来。

[连接第 3/4 讲](#)

Attention 的核心 score 计算，本质就是“Dot Product 批量化”。你以前学的向量点积现在终于进入 Transformer 的中心位置。

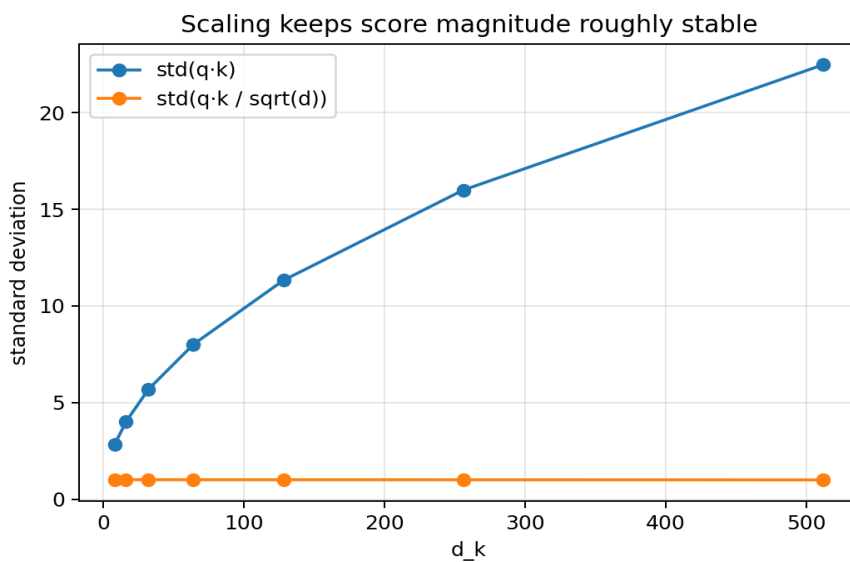
4. 为什么不能直接把 QK^T 送进 Softmax

如果 Q/K 每个分量都大致是零均值、方差 1 的随机变量，那么一个 Dot Product 是 d_k 个乘积的和。粗略地说：

$$\text{Var}(q \cdot k) \approx d_k$$

$$\text{Std}(q \cdot k) \approx \sqrt{d_k}$$

也就是说， d_k 越大，未经缩放的 score 典型幅度越大。Softmax 对数值差距非常敏感：输入差距一旦很大，概率会迅速接近 0 或 1。



为什么是 $\sqrt{d_k}$

因为 score 的标准差近似按 $\sqrt{d_k}$ 增长。除以 $\sqrt{d_k}$ 后，score 的典型尺度能保持在相近数量级。不是除以 d_k ，也不是为了“让和变平均”。

5. 第二步：Scaled Score

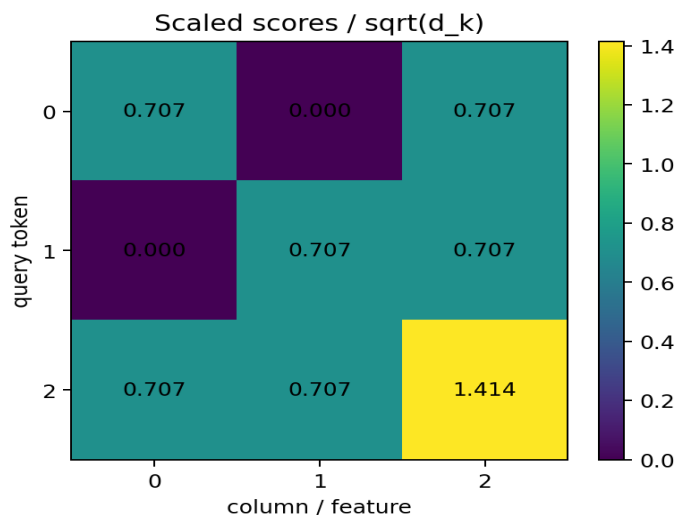
$$\text{ScaledScores} = QK^T / \sqrt{d_k}$$

本例 $d_k=2$ ：

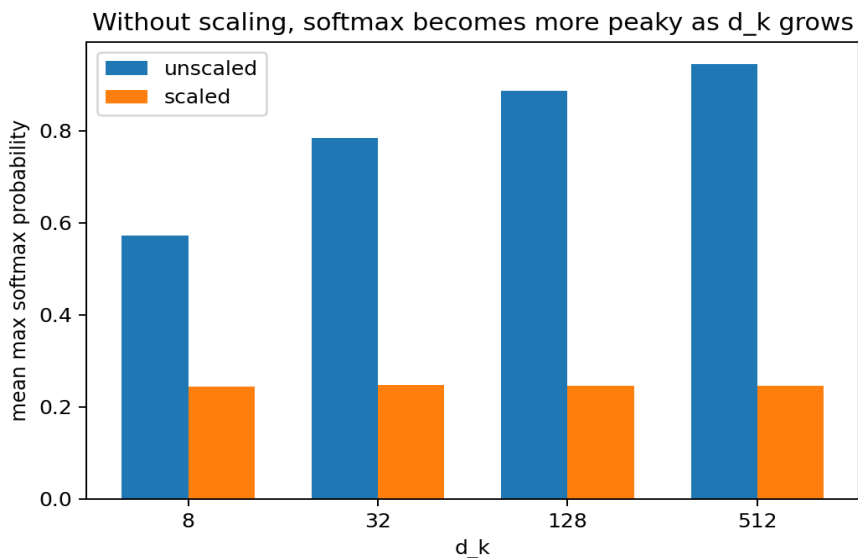
$$\sqrt{2} \approx 1.414$$

所以：

$$\text{Scaled} = [[0.707, 0, 0.707], [0, 0.707, 0.707], [0.707, 0.707, 1.414]]$$



Scaling 不改变 score 的相对排序，但会改变 Softmax 的“尖锐程度”。

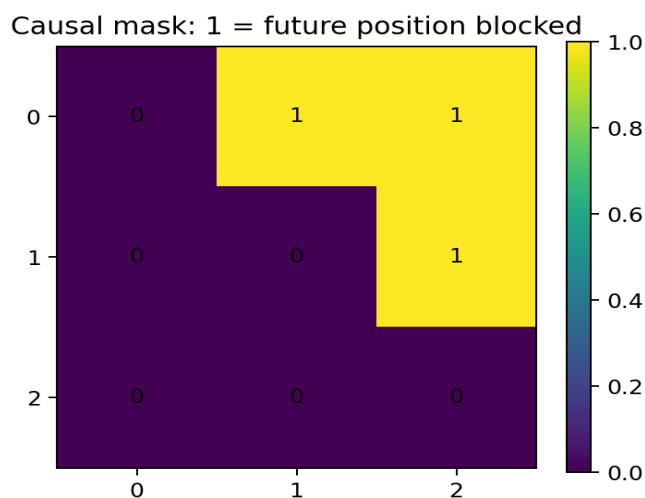


直觉

可以把它看成固定的 temperature 调节：未经缩放的大 score 更容易让 Softmax 过早接近 one-hot，从而梯度集中在极少数位置。

6. 第三步：Mask - 哪些 Key 根本不允许被看到

Decoder-only LLM 使用 Causal Mask：Token t 不能偷看未来 Token。对于 $T=3$ ：



常见实现不是直接删除未来位置，而是在 Softmax 前把对应 score 设成负无穷：

$$\text{masked_score}(i,j) = -\text{inf if } j > i$$

这样：

$$\exp(-\text{inf}) = 0$$

经过 Softmax 后未来位置权重严格为 0。

为什么 Mask
必须在 Softmax 前

如果先 Softmax 再把未来概率手工置 0，但不重新归一化，那么每行概率和不再等于 1。本讲实验专门验证了这个错误写法。

7. Causal Mask 后的 Score

本例 Mask 后：

[[0.707, -inf, -inf], [0,0.707,-inf], [0.707,0.707,1.414]]

注意第一行只能看 Token1；第二行能看 Token1-2；第三行能看 Token1-3。

这正对应 next-token prediction

训练中的因果约束：虽然整段序列可以并行计算，但每个位置的信息访问范围仍然受到 Mask 限制。

Query 位置	允许看的 Key
Token1	1
Token2	1,2
Token3	1,2,3

与第 51 讲连接 Causal Mask 约束信息方向，但并不强迫训练像 RNN 那样逐时间步执行。T×T score matrix 仍然可以一次计算。

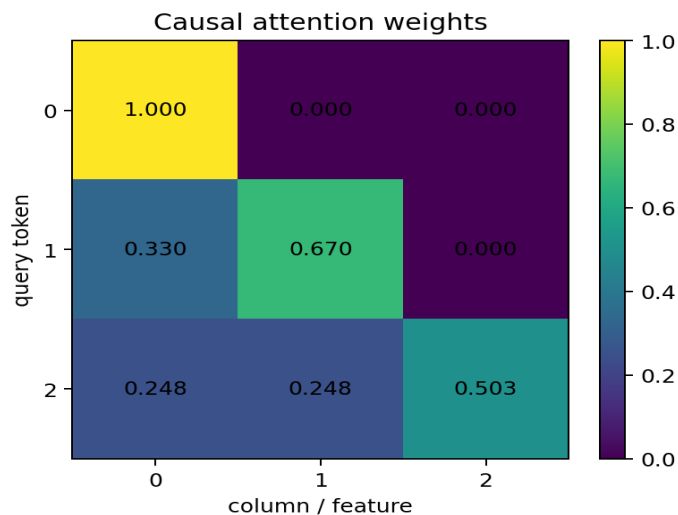
8. 第四步：对每一行做 Softmax

$$A = \text{Softmax}(\text{MaskedScores}, \text{dim}=\text{Key})$$

每一个 Query 都得到一行针对所有 Key 的权重分布。

本例 Causal Attention Weights 约为：

[[1,0,0], [0.330,0.670,0], [0.248,0.248,0.503]]



每一行都满足：

$$A[i,j] \geq 0, \sum_j A[i,j] = 1$$

Softmax 的轴

Attention 最常见的 Softmax 是沿 Key 维度做：固定一个 Query，让它在“可访问的 Key”之间分配总和为 1 的权重。

9. 第五步：Attention Weight @ V

$$\text{Output} = A @ V$$

A 的一行告诉我们当前 Query 应该从各个 Value 读取多少。

以 Token2 为例：

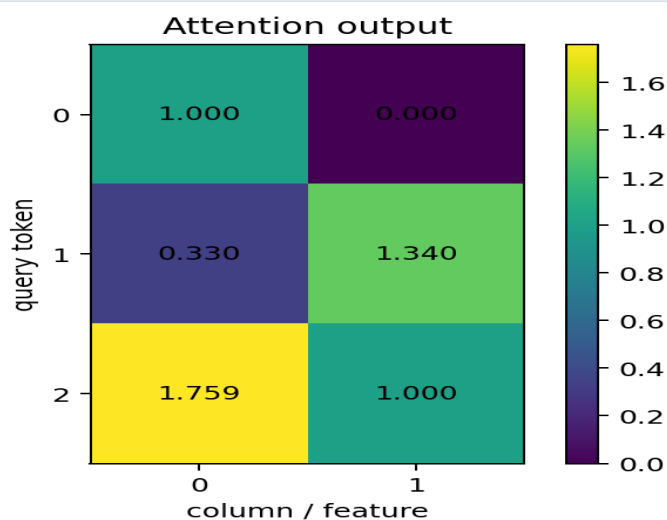
$$A_2 = [0.330, 0.670, 0]$$

$$V_1=[1,0], V_2=[0,2], V_3=[3,1]$$

因此：

$$o_2 = 0.330 * V_1 + 0.670 * V_2 + 0 * V_3$$

$$o_2 \approx [0.330, 1.340]$$



Attention
真正发生 Token
mixing 的地方

QKV Projection 只是逐 Token 特征变换； $A @ V$ 才把其他 Token 的 Value 加权汇入当前 Token 输出。这里第一次真正发生跨 Token 信息聚合。

10. 为什么说 Output 是 Value 的“加权和”

因为 Softmax Weight 非负且和为 1，所以每个输出向量都是可访问 Value 的加权组合。

$$o_i = \sum_j A[i,j] V_j$$

本例第三个 Token：

$$o_3 = 0.248 V_1 + 0.248 V_2 + 0.503 V_3$$

这与 RNN 的固定 Hidden State 压缩方式非常不同：Attention 在每一层、每个 Query、每个样本上都可以动态决定从哪些位置取多少信息。

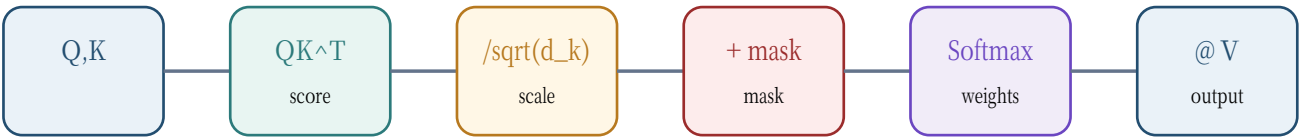
但不要把
Attention Weight
机械解释成因果
重要性

权重大说明该层该 Head 的 Value 聚合中占比大，但不能直接等价于“这个 Token 对最终输出的因果贡献最大”。网络还有多层、残差、MLP 和其他 Head。

11. 整条公式现在可以逐项读懂了

Attention(Q,K,V) = Softmax((QK^T)/sqrt(d_k)) V		
部分	作用	Shape（忽略 B/H）
QK^T	所有 Query-Key 匹配分数	(T,T)
/sqrt(d_k)	控制 score 尺度	(T,T)
+ Mask	阻断不允许访问的位置	(T,T)
Softmax	每个 Query 形成 Key 权重分布	(T,T)
@ V	按权重聚合 Value	(T,d_v)

Scaled Dot-Product Attention 的完整数据流



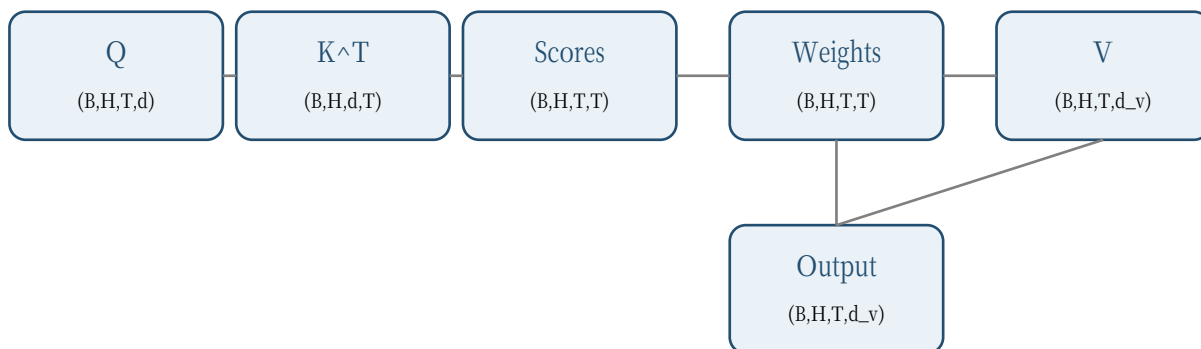
每一行 Softmax 都在 Key 维度上归一化，最后得到对 V 的加权和

到这里才算真正理解公式

不要把这一行当一个整体公式背。它其实是五个已经学过的基础操作按顺序连接：矩阵乘法 -> 缩放 -> Mask -> Softmax -> 矩阵乘法。

12. 加上 Batch 和 Head 以后，Shape 怎么走

Attention 的 Shape 主线



典型 Multi-Head 形式：

Q,K,V: (B,H,T,d_head)
Scores = Q @ K^T: (B,H,T,T)
Weights: (B,H,T,T)
Output: (B,H,T,d_v)

这里 transpose 的通常是最后两个维度：

```
scores = Q @ K.transpose(-2, -1)
scores = scores / math.sqrt(d_head)
weights = torch.softmax(scores, dim=-1)
out = weights @ V
```

Shape Debug

看到 Attention 报错时，第一检查 d_head 是否相等；第二检查 K 是否只转置最后两个轴；第三检查 Softmax 是否在最后的 Key 维。

13. Padding Mask 与 Causal Mask 不是同一个东西

Mask	解决的问题	典型含义
Causal Mask	不能看未来	位置 $j>i$ 被阻断
Padding Mask	不能把 PAD 当真实内容	PAD key 位置被阻断
自定义 Attention Mask	任务定义的可见性规则	例如局部窗口、结构约束

一个真实 batch 可能同时需要 Causal Mask 和 Padding Mask。实现上通常把它们组合成最终的可见性约束。

Mask 的语义一定要核对

不同框架的 bool mask 语义可能不同：有的 True 表示保留，有的 True 表示屏蔽。不要靠记忆猜，应该查当前 API 的定义并写最小数值测试。

14. 数值稳定性：Softmax 前为什么还要小心

第 17 讲已经学过稳定 Softmax：

$$\text{Softmax}(z) = \exp(z - \max(z)) / \sum \exp(z - \max(z))$$

Attention 里同样适用。Mask 常用 `-inf`，使被屏蔽位置在指数后变 0。现代框架会在 fused kernel 中处理 scaling、mask、softmax 等细节。

PyTorch 现在提供：

```
F.scaled_dot_product_attention(  
    query, key, value,  
    attn_mask=None,  
    dropout_p=0.0,  
    is_causal=True  
)
```

本讲实验把手写 causal attention 与该函数对照，最大绝对误差仅为浮点误差级别。

工程和数学分开
理解

底层 kernel 可以非常复杂，但你判断结果是否合理时，仍然应该能回到手写公式检查 Shape、Mask、Softmax 行和 Value 聚合。

15. Backward : Q、K、V 都会收到 Gradient

Attention Output 参与最终 Loss, 因此 Gradient 会沿计算图回到 Q、K、V, 再继续回到 $W_Q/W_K/W_V$ 和 X 。

Loss -> Output -> Weights/V -> Scores -> Q/K

实验中对 Attention 输出定义一个简单 MSE 类标量 Loss, Backward 后 Q、K、V 的 Gradient Norm 均非零。

Q/K 的梯度路径经过 Softmax 和 score ; V 的梯度主要经过最终加权和。三条路径的数学形式不同, 但都会被 Autograd 自动追踪。

Mask 对 Backward 的影响

被 Causal Mask 设为不可见的位置, 其 Softmax 权重为 0, 对应的直接 Attention 路径也被阻断。这里可以把 Mask 理解成 Forward 可见性约束, 同时改变 Backward 的有效计算图。

16. 为什么 Attention 能动态改变 “ 读取对象 ”

CNN 的 Kernel Weight 是固定参数；RNN 的状态转移 Weight 也是固定参数。Attention 中 $W_Q/W_K/W_V$ 同样是固定训练参数，但真正的 Attention Weight A 不是固定参数，而是由当前输入动态计算：

$$A(X) = \text{Softmax}(Q(X) K(X)^T / \sqrt{d_k})$$

所以不同句子、不同位置、不同 Layer、不同 Head 都可以产生不同 Attention Pattern。

这是 Attention 的关键能力之一	参数是学习出来的固定规则，但 “ 当前该看谁 ” 由输入内容动态决定。可以把它理解成 data-dependent routing / content-based addressing。
----------------------	---

这也是为什么 Attention 对序列关系建模比固定局部 Kernel 更灵活，但同时要付出 $T \times T$ score matrix 的计算与显存代价。

17. 常见误区与科研边界

误区	正确理解
Attention Weight 就是概率模型置信度	它只是当前 Query 对 Key 的归一化读取权重
除以 $\sqrt{d_k}$ 是为了让和变平均	核心是控制 score 方差/尺度
Causal Mask 会让训练逐 Token 执行	不会，整张 score matrix 可并行计算
Softmax 后再把未来权重置 0 也一样	若不重新归一化则不等价
Attention 一定能解决无限长上下文	标准 Attention 仍受上下文长度、 T^2 成本和模型能力限制
Attention 图能直接证明因果机制	通常只能作为证据之一，不能单独当因果证明
做 Ablation 时先 定位改了哪一步	Scale、Mask、score function、Softmax、Value aggregation、Head 结构是不同机制。论文说“改进 Attention ”时，先把改动放回本讲流水线再判断。

18. 与 LLM / Agent Systems 的直接连接

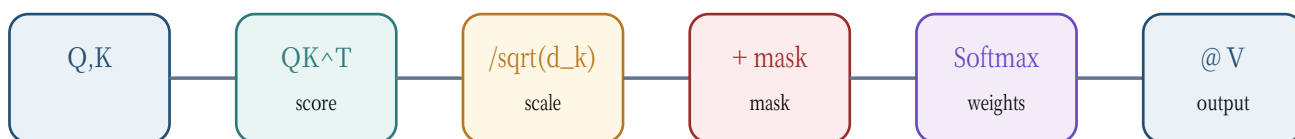
Decoder-only LLM 每一层都在重复类似 Attention 计算。长上下文、KV Cache、GQA/MQA、FlashAttention、Sparse Attention，本质上都与这条数据流的某一部分相关。

现代概念	与本讲哪一步最相关
KV Cache	复用历史 K/V，减少自回归生成重复投影/计算
FlashAttention	改变 Attention 的内存/计算实现，不改变数学结果目标
GQA / MQA	改变 Q head 与 K/V head 的组织和共享方式
Sparse / Local Attention	改变 Mask / 可见性模式
RoPE	在 score 计算前对 Q/K 注入位置结构
Attention Dropout	对 Attention Weight 路径施加随机正则化

在 Agent Systems 中，你往往不会自己改底层 Attention，但理解它能帮助你判断长上下文、记忆、KV Cache、上下文压缩和模型推理成本问题。

19. 本讲总装与 5 分钟自检

Scaled Dot-Product Attention 的完整数据流



每一行 Softmax 都在 Key 维度上归一化，最后得到对 V 的加权和

你现在应该能把下面一行完整口述出来：

$$O = \text{Softmax}((QK^T + \text{Mask})/\text{sqrt}(d_k)) V$$

自检：

- 1) QK^T 的每一个元素是什么意思？为什么 Shape 是 $T \times T$ ？
- 2) 为什么除以 $\text{sqrt}(d_k)$ ，而不是 d_k ？
- 3) 为什么 Causal Mask 要在 Softmax 前处理？
- 4) Softmax 应该沿哪个维度做？每一行为什么和为 1？
- 5) Attention Output 为什么是 Value 的加权和？
- 6) QKV Projection 和真正的 Token mixing 分别发生在哪一步？
- 7) 训练时 causal attention 为什么仍然可以并行计算整张矩阵？

下一讲

第 55 讲进入 Mask 与 Multi-Head Attention：把一个大的 Attention 子空间拆成多个 Head，并详细追踪 reshape、transpose、concat 和 output projection 的 Shape。